```
# @title
!pip install transformers
!pip install datasets
!pip install scikit-learn
!pip install pandas
!pip install matplotlib
!pip install seaborn
!pip install torch
```

```
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in /usr/local/lib/py
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in /usr/local/
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in /usr/local/li
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/di
Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.:
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/(
```

```
import pandas as pd

fake = pd.read_csv("Fake.csv")
true = pd.read_csv("True.csv")

fake["label"] = 1
true["label"] = 0
df = pd.concat([fake, true])
df["content"] = df["title"] + " " + df["text"]
df = df[["content","label"]]
df
```

	content	label
0	Donald Trump Sends Out Embarrassing New Year'...	1
1	Drunk Bragging Trump Staffer Started Russian ...	1
2	Sheriff David Clarke Becomes An Internet Joke...	1
3	Trump Is So Obsessed He Even Has Obama's Name...	1
4	Pope Francis Just Called Out Donald Trump Dur...	1
...	...	...
21412	'Fully committed' NATO backs new U.S. approach...	0
21413	LexisNexis withdrew two products from Chinese ...	0
21414	Minsk cultural hub becomes haven from authorit...	0
21415	Vatican upbeat on possibility of Pope Francis ...	0
21416	Indonesia to buy $1.14 billion worth of Russia...	0

44898 rows × 2 columns

Next steps: Generate code with df New interactive sheet

```
df.head()
```

	content	label
0	Donald Trump Sends Out Embarrassing New Year'...	1
1	Drunk Bragging Trump Staffer Started Russian ...	1
2	Sheriff David Clarke Becomes An Internet Joke...	1
3	Trump Is So Obsessed He Even Has Obama's Name...	1
4	Pope Francis Just Called Out Donald Trump Dur...	1

Next steps: Generate code with df New interactive sheet

```
df.shape
```

```
(44898, 2)
```

```
df.columns
```

```
Index(['content', 'label'], dtype='object')
```

```
df.isnull().sum()
```

	0
content	0
label	0

dtype: int64

```
df['label'].value_counts()
```

label	count
1	23481
0	21417

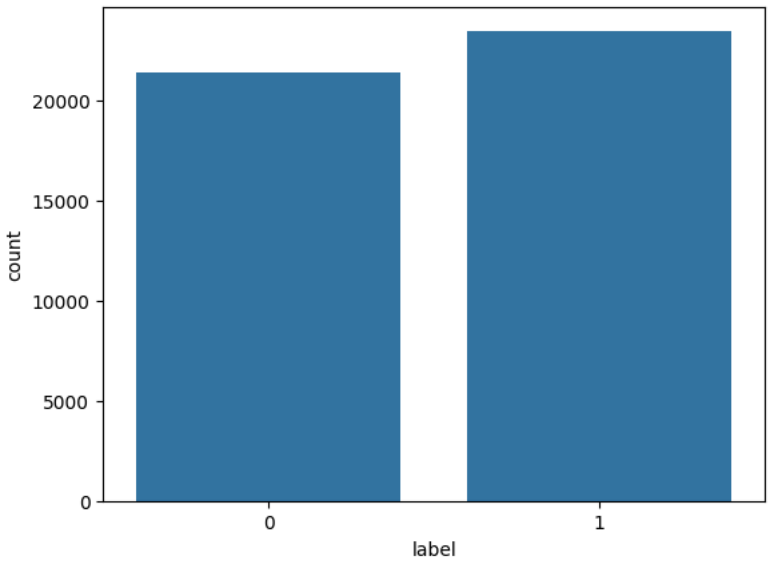
dtype: int64

```
df.head(10)
```

	content	label
0	Donald Trump Sends Out Embarrassing New Year'...	1
1	Drunk Bragging Trump Staffer Started Russian ...	1
2	Sheriff David Clarke Becomes An Internet Joke...	1
3	Trump Is So Obsessed He Even Has Obama's Name...	1
4	Pope Francis Just Called Out Donald Trump Dur...	1
5	Racist Alabama Cops Brutalize Black Boy While...	1
6	Fresh Off The Golf Course, Trump Lashes Out A...	1
7	Trump Said Some INSANELY Racist Stuff Inside ...	1
8	Former CIA Director Slams Trump Over UN Bully...	1
9	WATCH: Brand-New Pro-Trump Ad Features So Muc...	1

Next steps: Generate code with df New interactive sheet

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.countplot(x='label', data=df)
plt.show()
```




```
import torch
from transformers import AutoTokenizer
from sklearn.model_selection import train_test_split
```

```
train_texts, val_texts, train_labels, val_labels = train_test_split(
    df["content"],
    df["label"],
    test_size=0.2,
    random_state=42
)
```

```
print(len(train_texts))
print(len(val_texts))
```

```
35918
8980
```

```
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")
```

```
train_encodings = tokenizer(
    train_texts.tolist(),
    truncation=True,
    padding="max_length",
    max_length=256
)

val_encodings = tokenizer(
    val_texts.tolist(),
    truncation=True,
    padding="max_length",
    max_length=256
)
```

```
print(train_encodings['input_ids'][0])
```

```
[101, 4977, 1005, 1055, 9413, 16168, 2319, 2758, 1057, 1012, 1055, 1012, 6744,
```

```
class FakeNewsDataset(torch.utils.data.Dataset):

    def __init__(self, encodings, labels):
        self.encodings = encodings
        self.labels = labels

    def __getitem__(self, idx):
        item = {key: torch.tensor(val[idx]) for key, val in self.encodings.it
        item["labels"] = torch.tensor(self.labels[idx], dtype=torch.long)
        return item

    def __len__(self):
        return len(self.labels)
```

```
train_dataset = FakeNewsDataset(train_encodings, train_labels.tolist())
val_dataset = FakeNewsDataset(val_encodings, val_labels.tolist())
```

```
sample = train_dataset[0]

for key, value in sample.items():
    print(key, value.shape)
```

```
input_ids torch.Size([256])
token_type_ids torch.Size([256])
attention_mask torch.Size([256])
labels torch.Size([])
```

```
import torch
from transformers import AutoModelForSequenceClassification
from transformers import TrainingArguments, Trainer
from sklearn.metrics import accuracy_score, precision_recall_fscore_support
```

```
model = AutoModelForSequenceClassification.from_pretrained(
    "distilbert-base-uncased",
    num_labels=2
)
```

```
Loading weights: 100%   100/100 [00:00<00:00, 308.05it/s, Materializing param=distilbert.transform

DistilBertForSequenceClassification LOAD REPORT from: distilbert-base-uncased
Key                      | Status     |
-------------------------+------------+-
vocab_layer_norm.bias    | UNEXPECTED |
vocab_projector.bias     | UNEXPECTED |
vocab_transform.bias     | UNEXPECTED |
vocab_layer_norm.weight  | UNEXPECTED |
vocab_transform.weight   | UNEXPECTED |
pre_classifier.weight    | MISSING    |
classifier.weight        | MISSING    |
classifier.bias          | MISSING    |
pre_classifier.bias      | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture;
```

```
def compute_metrics(eval_pred):

    logits, labels = eval_pred
    predictions = logits.argmax(axis=1)

    precision, recall, f1, _ = precision_recall_fscore_support(
        labels, predictions, average="binary"
    )

    acc = accuracy_score(labels, predictions)

    return {
        "accuracy": acc,
        "f1": f1,
        "precision": precision,
        "recall": recall
    }
```

```
training_args = TrainingArguments(
    output_dir="./results",

    learning_rate=3e-5,

    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,

    num_train_epochs=3,

    eval_strategy="epoch",

    save_strategy="epoch",

    logging_dir="./logs",

    load_best_model_at_end=True,

    weight_decay=0.01
)
```

```
`logging_dir` is deprecated and will be removed in v5.2. Please set `TENSORBOA
```

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
    compute_metrics=compute_metrics
)
```

```
print(train_dataset[0])
```

```
{'input_ids': tensor([  101,  4977,  1005,  1055,  9413, 16168,  2319,  2758,
         1055,  1012,  6744,  3247, 12517, 21112,  2006,  2375,  7571,  1006,
        26665,  1007,  1011,  5037,  2343, 28117, 10139,  2361,  9413, 16168,
         2319,  2056,  2006,  9432,  2008,  1057,  1012,  1055,  1012,  2343,
         6221,  8398,  1055, 15140,  3247,  2000,  6807,  6744,  2004,  1996,
         3007,  1997,  3956,  2001, 12517, 14353,  2006,  2248,  4277,  1012,
         9413, 16168,  2319,  4092,  1999,  7571,  2044,  7566,  2007,  3539,
         2704, 13573, 24529, 11514,  8180,  1010,  2036,  2056,  4977,  2359,
         2000,  2156,  1037,  9879,  5576,  2006,  1996,  2479,  1997,  9719,
         1010,  2021,  2056,  3306, 18543,  2015,  2020,  9992,  7566,  1012,
          102,     0,     0,     0,     0,     0,     0,     0,     0,     0,
            0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
            0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
            0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
            0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
            0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
            0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
            0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
            0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
            0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
            0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
```

```
            0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
            0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
            0,     0,     0,     0,     0,     0]), 'token_type_ids': tensor([
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]), 'attention_mask': te
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1
        1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]), 'labels': tensor(0)}
```

```
print(df["content"].iloc[0])
```

```
 Donald Trump Sends Out Embarrassing New Year’s Eve Message; This is Disturbin
```

```
trainer.train()
```

[6735/6735 47:12, Epoch 3/3]

Epoch	Training Loss	Validation Loss	Accuracy	F1	Precision	Recall
1	0.003469	0.000021	1.000000	1.000000	1.000000	1.000000
2	0.000690	0.000004	1.000000	1.000000	1.000000	1.000000
3	0.000004	0.000002	1.000000	1.000000	1.000000	1.000000

```
Writing model shards: 100%                                    1/1 [00:02<00:00,  2.97s/it]
Writing model shards: 100%                                    1/1 [00:11<00:00, 11.77s/it]
Writing model shards: 100%                                    1/1 [00:25<00:00, 25.84s/it]
There were missing keys in the checkpoint model loaded: ['distilbert.embedding
There were unexpected keys in the checkpoint model loaded: ['distilbert.embedd
TrainOutput(global_step=6735, training_loss=0.004216526687633019, metrics=
```

```
trainer.save_model("fake_news_model")
tokenizer.save_pretrained("fake_news_model")
```

```
Writing model shards: 100%                                    1/1 [00:20<00:00, 20.20s/it]
('fake_news_model/tokenizer_config.json', 'fake_news_model/tokenizer.json')
```

```
from google.colab import drive
```

```
drive.mount('/content/drive')

trainer.save_model("/content/drive/MyDrive/fake_news_model")
tokenizer.save_pretrained("/content/drive/MyDrive/fake_news_model")
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call
Writing model shards: 100%                                    1/1 [01:46<00:00, 106.30s/it]
('/content/drive/MyDrive/fake_news_model/tokenizer_config.json',
 '/content/drive/MyDrive/fake_news_model/tokenizer.json')
```

```
trainer.evaluate()
```

[562/562 00:58]

```
{'eval_loss': 0.6869763731956482,
 'eval_model_preparation_time': 0.0538,
 'eval_accuracy': 0.6453229398663697,
 'eval_f1': 0.6118220597196831,
 'eval_precision': 0.722926267281106,
 'eval_recall': 0.5303190365518698,
 'eval_runtime': 59.592,
 'eval_samples_per_second': 150.691,
 'eval_steps_per_second': 9.431}
```

```
predictions = trainer.predict(val_dataset)
```

```
preds = predictions.predictions.argmax(axis=1)
```

```
from sklearn.metrics import classification_report

print(classification_report(val_labels, preds))
```

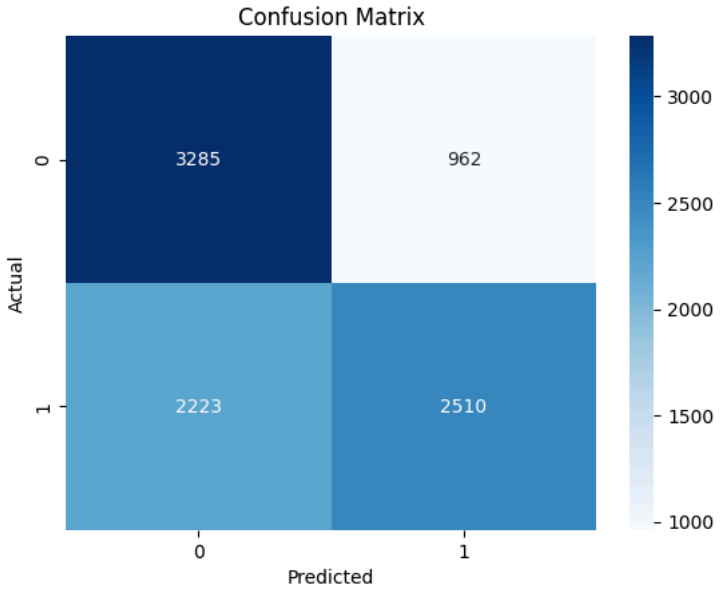
```
              precision    recall  f1-score   support

           0       0.60      0.77      0.67      4247
           1       0.72      0.53      0.61      4733

    accuracy                           0.65      8980
   macro avg       0.66      0.65      0.64      8980
weighted avg       0.66      0.65      0.64      8980
```

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(val_labels, preds)
print(cm)
```

```
[[3285  962]
 [2223 2510]]
```

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
```

```
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```




```
wrong_predictions = []

for text, true_label, pred_label in zip(val_texts, val_labels, preds):
    if true_label != pred_label:
        wrong_predictions.append((text, true_label, pred_label))

len(wrong_predictions)
```

```
3185
```

```
!pip install gradio
```

```
Requirement already satisfied: gradio in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: aiofiles<25.0,>=22.0 in /usr/local/lib/python3.
Requirement already satisfied: anyio<5.0,>=3.0 in /usr/local/lib/python3.12/di
Requirement already satisfied: brotli>=1.1.0 in /usr/local/lib/python3.12/dist
Requirement already satisfied: fastapi<1.0,>=0.115.2 in /usr/local/lib/python3
Requirement already satisfied: ffmpy in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: gradio-client==1.14.0 in /usr/local/lib/python3
Requirement already satisfied: groovy~=0.1 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: httpx<1.0,>=0.24.1 in /usr/local/lib/python3.12
Requirement already satisfied: huggingface-hub<2.0,>=0.33.5 in /usr/local/lib/
Requirement already satisfied: jinja2<4.0 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: markupsafe<4.0,>=2.0 in /usr/local/lib/python3.
Requirement already satisfied: numpy<3.0,>=1.0 in /usr/local/lib/python3.12/di
Requirement already satisfied: orjson~=3.0 in /usr/local/lib/python3.12/dist-p
```

```
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: pandas<3.0,>=1.0 in /usr/local/lib/python3.12/d
Requirement already satisfied: pillow<12.0,>=8.0 in /usr/local/lib/python3.12/
Requirement already satisfied: pydantic<=2.12.3,>=2.0 in /usr/local/lib/python
Requirement already satisfied: pydub in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: python-multipart>=0.0.18 in /usr/local/lib/pyth
Requirement already satisfied: pyyaml<7.0,>=5.0 in /usr/local/lib/python3.12/d
Requirement already satisfied: ruff>=0.9.3 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: safehttpx<0.2.0,>=0.1.6 in /usr/local/lib/pytho
Requirement already satisfied: semantic-version~=2.0 in /usr/local/lib/python3
Requirement already satisfied: starlette<1.0,>=0.40.0 in /usr/local/lib/python
Requirement already satisfied: tomlkit<0.14.0,>=0.12.0 in /usr/local/lib/pytho
Requirement already satisfied: typer<1.0,>=0.12 in /usr/local/lib/python3.12/d
Requirement already satisfied: typing-extensions~=4.0 in /usr/local/lib/python
Requirement already satisfied: uvicorn>=0.14.0 in /usr/local/lib/python3.12/di
Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: websockets<16.0,>=13.0 in /usr/local/lib/python
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/pyth
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: filelock>=3.10.0 in /usr/local/lib/python3.12/d
Requirement already satisfied: hf-xet<2.0.0,>=1.4.2 in /usr/local/lib/python3.
Requirement already satisfied: tqdm>=4.42.1 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dis
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/pytho
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-pa
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call
```

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification

model = AutoModelForSequenceClassification.from_pretrained("/content/drive/My
tokenizer = AutoTokenizer.from_pretrained("/content/drive/MyDrive/fake_news_m
```

```
Loading weights: 100%  104/104 [00:00<00:00, 352.35it/s, Materializing param=pre_classifier.weig
```

```
model.eval()
```

```
DistilBertForSequenceClassification(
  (distilbert): DistilBertModel(
    (embeddings): Embeddings(
```

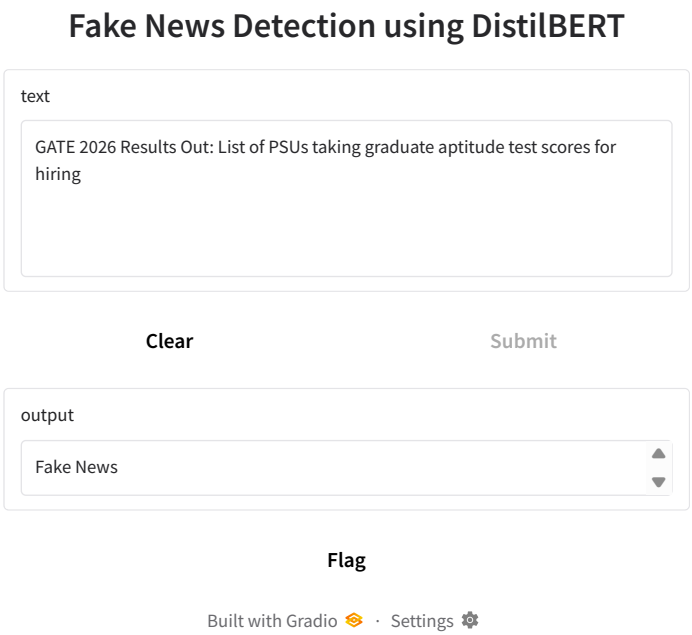
```
      (word_embeddings): Embedding(30522, 768, padding_idx=0)
      (position_embeddings): Embedding(512, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (transformer): Transformer(
      (layer): ModuleList(
        (0-5): 6 x TransformerBlock(
          (attention): DistilBertSelfAttention(
            (q_lin): Linear(in_features=768, out_features=768, bias=True)
            (k_lin): Linear(in_features=768, out_features=768, bias=True)
            (v_lin): Linear(in_features=768, out_features=768, bias=True)
            (out_lin): Linear(in_features=768, out_features=768, bias=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
          (sa_layer_norm): LayerNorm((768,), eps=1e-12,
elementwise_affine=True)
          (ffn): FFN(
            (dropout): Dropout(p=0.1, inplace=False)
            (lin1): Linear(in_features=768, out_features=3072, bias=True)
            (lin2): Linear(in_features=3072, out_features=768, bias=True)
            (activation): GELUActivation()
          )
          (output_layer_norm): LayerNorm((768,), eps=1e-12,
elementwise_affine=True)
        )
      )
    )
  )
  (pre_classifier): Linear(in_features=768, out_features=768, bias=True)
  (classifier): Linear(in_features=768, out_features=2, bias=True)
  (dropout): Dropout(p=0.2, inplace=False)
)
```

```
import torch

def predict(text):
    inputs = tokenizer(
        text,
        return_tensors="pt",
        truncation=True,
        padding=True,
        max_length=256
    )

    outputs = model(**inputs)
    probs = torch.nn.functional.softmax(outputs.logits, dim=-1)

    pred = torch.argmax(probs).item()

    if pred == 0:
        return "Real News"
    else:
        return "Fake News"
```

```
import gradio as gr

interface = gr.Interface(
```

```
    fn=predict,
    inputs=gr.Textbox(lines=5, placeholder="Enter news text here"),
    outputs="text",
    title="Fake News Detection using DistilBERT"
)

interface.launch(share=True)
```

```
Colab notebook detected. To show errors in colab notebook, set debug=True in l
* Running on public URL: https://2eef65ed85defa6e96.gradio.live

This share link expires in 1 week. For free permanent hosting and GPU upgrades
```



Next steps: Deploy to Cloud Run